«Мы можем видеть лишь небольшое расстояние впереди, но уже видим, как много там нужно сделать.»

— Алан Тьюринг

> Куратор составляет опись Машины из конечного списка состояний, переходов и алфавита ленты. Опись сама является записью, поэтому её можно предъявить Машине и спросить о её собственном поведении.
>
> Одни запросы Машина разбирает за время, растущее как степень длины входа, а для других известен только перебор, растущий экспоненциально.

Задача 1: Машина Тьюринга — инкремент

База

Куратор приспосабливает Машину под скучную, но нужную работу: прибавлять единицу к двоичной записи числа на ленте. Числа записаны младшим разрядом вперёд, а справа от записи идут пустые ячейки.

> Машина Тьюринга задаётся семёркой $(Q, \Gamma, \Sigma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$: Q — состояния, Γ — алфавит ленты с пустой ячейкой $\square$, $\Sigma \subseteq \Gamma \setminus \{\square\}$ — входной алфавит. Функция переходов частична: $\delta : Q \times \Gamma \to Q \times \Gamma \times \{\mathrm{L}, \mathrm{R}\}$, и запись $\delta(q, a) = (q', b, \mathrm{L})$ означает, что в состоянии q, читая a, машина пишет b, переходит в q' и сдвигает головку влево. Если значения δ на паре (состояние, символ) нет, машина в этой паре останавливается.
>
> Конфигурация записывается как uqv: слово uv — содержимое ленты, q — состояние, головка стоит на первом символе v. Шаг машины переводит одну конфигурацию в следующую. Машина принимает слово, если останавливается в состоянии q_{accept}, и отвергает, если останавливается в состоянии q_{reject}.

1. Напишите состояния и переходы машины (головка стартует на младшем разряде, он лежит слева) и объясните, как переносится единица.
2. Проследите её работу на записи «1101» (число 11), выписывая конфигурации на каждом шаге.
3. Проследите работу на записи «111» (число 7) и приведите запись, которая остаётся на ленте, когда перенос проходит через все разряды.

Задача 2: Конфигурации и язык $a^n b^n$

База

Машина проверяет пароли к хранилищу, и годный пароль устроен как n букв a, за которыми идут n букв b. Куратор хочет видеть её работу по шагам и знать, чего стоит один прогон.

1. Постройте машину: помечайте первую непомеченную a, идите направо, помечайте первую непомеченную b, возвращайтесь и повторяйте. В конце проверьте, что непомеченных букв не осталось и порядок a^*b^* не нарушен.
2. Выпишите последовательность конфигураций построенной машины на входе «aaabbb» до остановки.
3. Сколько шагов она сделает на слове $a^n b^n$? Выразите через n.
4. Что произойдёт на слове «aabbb» (две a, три b)? Машина должна отвергнуть слово: проследите прогон до остановки и укажите шаг, на котором это видно.

Задача 3: Распознавание и разрешение

Челлендж

Куратору нужен полный список слов, принимаемых машиной. Список можно строить, прогоняя слова через машину, но на некоторых словах машина работает вечно и ответа не даёт.

1. Докажите, что если язык L перечислим и его дополнение тоже перечислимо, то L разрешим. Постройте решающую машину, которая по очереди делает шаг первого перечислителя, шаг второго, и объясните, почему она всегда останавливается с верным ответом.
2. Приведите конкретный язык, который перечислим, но не разрешим: предъявите распознающую его машину и докажите, что решающей нет.
3. Постройте машину, которая распознаёт разрешимый язык, но на словах дополнения не останавливается никогда. Объясните, почему перечислить дополнение прогоном слов через эту машину не удаётся, хотя язык разрешим.

Задача 4: Проблема остановки

Челлендж

Куратор хочет программу, которая по описанию *любой* машины и *любому* входу отвечает, остановится ли она. Если бы такая программа была, она решала бы все вопросы о поведении машин сразу, и именно это, как показал Тьюринг, невозможно.

1. Предположите, что решатель H остановки существует, и постройте машину D, которая на собственном коде делает обратное предсказанию H. Покажите, что ответ H на пару (D, D) приводит к противоречию, и выведите, что решателя нет.
2. Укажите шаг рассуждения, где используется допущение «программа всегда отвечает», и что ломается, если разрешить ей не отвечать.
3. Соберите таблицу: строки — машины Тьюринга, столбцы — слова, в клетке — принимает ли машина слово. Постройте по диагонали этой таблицы язык, не совпадающий ни с одной её строкой, и объясните, почему такая же схема даёт несчётность $\mathbb{R}$ (там строки — числа, столбцы — разряды).

Задача 5: Машины Тьюринга счётны

База

Каждая машина Тьюринга — конечный набор переходов, а любую конечную запись можно переписать строкой символов.

1. Занумеруйте все машины Тьюринга натуральными числами: опишите кодирование машины словом над конечным алфавитом и обоснуйте, почему слов счётно.
2. Сравните число машин с числом всех языков над одним алфавитом и сделайте вывод о существовании языков, которые никакая машина не распознаёт.
3. Разделите языки на разрешимые, распознаваемые и нераспознаваемые и укажите, какая из трёх групп несчётна. Как это согласуется с существованием языков без распознающей машины?

Задача 6: Сводимость

База

Сведение — это способ переписать одну задачу в другую так, что ответ на вторую даёт ответ на первую. Именно так доказывают неразрешимость: если бы решалась вторая задача, решалась бы и неразрешимая первая.

1. Сведите проблему остановки к вопросу «пуст ли язык машины» (не принимает ни одного слова). По машине M и слову w постройте машину M', язык которой пуст тогда и только тогда, когда M на w не останавливается, и объясните, как из решателя для пустоты языка получился бы решатель остановки.
2. Сведите вопрос «принимает ли машина M слово w» к вопросу «останавливается ли машина на входе x» и опишите, как из решателя остановки получается решатель для приёма слова.
3. Сведение годится только в одну сторону: приведите сведение от разрешимой задачи к неразрешимой и объясните, почему оно ничего не доказывает.

Задача 7: Теорема Райса

Челлендж

Куратор хочет по одному лишь описанию машины понимать её поведение: останавливается ли она ровно на одном входе, печатает ли хоть один символ. По теореме Райса любое нетривиальное свойство вычислимой функции неразрешимо.

1. Докажите, что по описанию машины нельзя решить, останавливается ли она ровно на одном входе. Укажите, почему это свойство вычислимой функции, и сведите к нему проблему остановки.
2. Докажите, что вопрос «печатает ли машина хоть один символ» тоже неразрешим, сведя его к проблеме остановки. Это свойство запуска, а не вычислимой функции, поэтому теорема Райса к нему напрямую не применима.
3. Приведите пример *разрешимого* свойства машин и объясните, почему оно разрешимо, а предыдущие — нет.

Задача 8: Сертификаты и верификация

База

Куратору нужно понять, есть ли в сети хранилищ вершинное покрытие размера не более k, то есть набор узлов, задевающий каждое соединение. Проверять «да» легко, если кто-то предъявил готовое покрытие, и вся трудность в том, чтобы его *найти*.

> Верификатор — машина, которая получает слово w и сертификат c, работает за время $O(n^d)$ и принимает или отвергает пару. Здесь n — длина w, а d — постоянная. Язык L лежит в классе NP, если каждое слово $w \in L$ имеет сертификат длины не больше $O(n^d)$, который верификатор принимает, а слова $w \notin L$ не принимаются ни с каким сертификатом. Класс P состоит из языков, которые разрешаются детерминированной машиной за время $O(n^d)$.

1. Опишите *сертификат* и *проверяющую* программу, которая за полиномиальное время убеждается, что предъявленное покрытие задевает все рёбра и имеет размер $\leq k$. Оцените время проверки через число вершин и рёбер.

2. Покажите, что и задача «есть ли покрытие размера $\leq k$», и задача «есть ли покрытие *ровно* k» имеют полиномиальный сертификат. Объясните, чем различаются их сертификаты, и почему наличие сертификата ничего не говорит о сложности *поиска*.
3. Докажите, что класс задач с полиномиальным сертификатом содержит класс задач, решаемых за полиномиальное время, и укажите, что здесь служит сертификатом.

Задача 9: Недетерминированные машины

База

Машине можно разрешить *угадывать*, и тогда в каждой развилке она идёт по всем вариантам сразу, а не перебирает их по одному.

1. Постройте недетерминированную машину, которая за полиномиальное время угадывает вершинное покрытие размера k и проверяет его. Оцените число шагов угадывания через k и число вершин.
2. Оцените, во что обходится детерминированное моделирование такого угадывания: выразите время через k и число вершин.
3. Докажите, что по распознаваемым языкам недетерминированные машины равносильны детерминированным: укажите, как детерминированная машина повторяет дерево вычислений.

Задача 10: Сведение SAT к 3-SAT

Челлендж

Любую логическую формулу можно записать в конъюнктивной нормальной форме (КНФ), но на практике удобнее, когда в каждом дизъюнкте ровно три литерала.

1. Опишите преобразование произвольной КНФ в 3-КНФ: длинный дизъюнкт разбейте вспомогательными переменными, короткий дополните. Докажите в обе стороны, что исходная формула выполнима тогда и только тогда, когда выполним результат, и оцените число новых переменных и дизъюнктов через длину исходной формулы.
2. Объясните, почему из NP-полноты SAT и этого сведения следует NP-полнота «3-SAT».
3. Выпишите цепочку сведений SAT $\to$ 3-SAT $\to$ вершинное покрытие и объясните, почему достаточно доказать NP-полноту одной задачи, а остальные получать сведениями.

Задача 11: Сведение 3-SAT к вершинному покрытию

Челлендж

Куратор сводит логическую задачу к задаче о графе: по формуле φ с n переменными и m дизъюнктами он строит сеть хранилищ и число k так, что покрытие размера k существует тогда и только тогда, когда φ выполнима.

1. Опишите конструкцию. Каждой переменной x_i соответствует ребро из двух «значений» x_i и $\neg x_i$. Каждому дизъюнкту соответствует треугольник из трёх литеральных вершин, каждая соединена со своим «значением». Покажите, почему при такой конструкции $k = n + 2m$.
2. Покажите эквивалентность подробно, разбирая обе стороны: из выполнимой φ постройте покрытие, а из покрытия — выполняющий набор значений.
3. Подсчитайте число вершин и рёбер полученного графа через n и m и подтвердите полиномиальность сведения.

Задача 12: P vs NP — что на кону

База

Классы P и NP совпадают или нет, неизвестно, и вопрос открыт больше полувека. При равенстве классов всё, что можно быстро проверить, можно было бы и быстро найти.

1. Перечислите три практических последствия равенства $P = \text{NP}$: что станет с шифрованием, оптимизацией, автоматическим доказательством теорем.
2. Объясните, почему NP-полнота — плохая новость, но не приговор: приведите пример, когда NP-полную задачу всё равно решают на практике (и чем).
3. Объясните, почему из $P = \text{NP}$ следует $\text{NP} = \text{coNP}$, и приведите задачу из coNP, для которой полиномиальный алгоритм неизвестен.

Задача 13: Двусторонние сертификаты

Челлендж

Есть задачи, где и «да», и «нет» обосновываются коротким сертификатом. Проверка простоты числа — одна из них.

1. Для проверки простоты числа приведите оба коротких сертификата. Сертификат простоты: разложение $n - 1$ на простые множители и такое g, что $g^{n-1} \equiv 1 (\mathrm{mod}\, n)$, а $g^{(n-1)/p} \neq 1 (\mathrm{mod}\, n)$ для каждого простого p из разложения. Каждый простой множитель снабжается своим сертификатом. Сертификат составности: нетривиальный делитель. Предъявите сертификат простоты для $n = 1237$ и сертификат составности для $n = 1729$, а затем объясните, почему для составного числа условия сертификата простоты выполниться не могут.
2. Докажите, что если задача из класса $\text{NP} \cap \text{coNP}$ NP-полна, то $\text{NP} = \text{coNP}$.
3. Приведите пример ещё одной задачи с «двусторонним сертификатом» и выпишите оба сертификата. Берите задачу не из класса P: иначе сертификат есть, но пример не показателен для $\text{NP} \cap \text{coNP}$.

Задача 14: Цена вычислений

Челлендж

Куратору нужно знать, во что на самом деле обходится алгоритм.

1. Оцените, сколько примерно операций нужно для перебора 2^n при $n = 100$ и для алгоритма за n^3 при $n = 10^6$, и во сколько раз они различаются.
2. Найдите, при каком n перебор 2^n операций перестаёт укладываться в час на машине с 10^9 операций в секунду, и при каком n это происходит с алгоритмом за n^3. Сравните оба порога.
3. Объясните, почему многоленточная машина не мощнее одноленточной, но может быть быстрее по времени, и оцените, во сколько раз.

Необязательные бонусные задачи

> Бонусные задачи не входят в обязательный минимум. Они заметно сложнее основных и требуют либо программирования, либо самостоятельного обобщения материала. Оценивается в них прежде всего объяснение модели, её корректности и границ применимости.

Задача A: Минимальные и универсальные машины Тьюринга

Бонус

Куратору мало гонять отдельные машины, и он хочет самые компактные из них и одну, исполняющую все остальные.

1. *Модель — часть задачи.* Эмулятор (лента, состояния, функция перехода) — отправная точка: возьмите готовый симулятор или реализуйте заново шаг и запуск до остановки. При превышении предела шагов останавливайтесь с пометкой «не завершилась за лимит».
2. Минимизация: для построенных выше машин — инкремента и распознавателя $a^n b^n$ — попробуйте сократить число состояний слиянием неразличимых состояний или перебором машин с меньшим числом состояний по возрастанию размера. Сведите в таблицу размеры машин до и после минимизации, объясните, почему полный перебор меньших машин не даёт алгоритма минимизации, и укажите, как такой перебор ограничивают на практике.
3. Универсальность: закодируйте машину строкой (состояния и переходы) и постройте машину, которая по коду и входу воспроизводит работу закодированной машины. Сверьте прямой и универсальный прогон: вердикты и итоговые ленты должны совпадать.
4. Оформите отчёт: модель, таблица минимизации, кодирование и универсальная машина, ограничения.

Задача B: Полиномиальная иерархия и PSPACE

Бонус

Классы P и NP из задачи 12 стоят в начале лестницы сложности, и выше идут $\Sigma_1 = \mathrm{NP}$, $\Pi_1 = \mathrm{coNP}$, $\Sigma_2 = \mathrm{NP}^{\mathrm{NP}}$, Π_2 и так далее, вплоть до PSPACE.

1. Определите классы Σ_k и Π_k через NP с оракулом (или через чередование кванторов) и выпишите первые три уровня.
2. Докажите, что $\Sigma_1 = \mathrm{NP}$, $\Pi_1 = \mathrm{coNP}$, и включения $\Sigma_k \subseteq \Pi_{k+1}$, $\Pi_k \subseteq \Sigma_{k+1}$.
3. Приведите пример задачи из Σ_2, то есть задачи со схемой «существует …, для всех …»: например, существует ли раскраска вершин графа в три цвета, при которой нет треугольника со всеми тремя цветами.
4. Объясните, почему PSPACE содержит всю иерархию, и приведите пример PSPACE-полной задачи: например, выигрывает ли первый игрок в обобщённую географию, или выполнима ли формула с чередующимися кванторами (TQBF).
5. Оформите отчёт: определения через кванторы, цепочка включений, примеры с позиций «существует/для всех».

Задача C: Почему диагональ не доказывает $P \neq \mathrm{NP}$ Бонус

Диагональный приём из задачи 4 отделяет перечислимое от неразрешимого, и куратор проверяет, где эта лестница обрывается.

1. Укажите, что мешает диагональному аргументу (как в проблеме остановки) отделить P от NP: какую машину пришлось бы строить и почему она выходит за полиномиальное время.
2. Сформулируйте теорему об иерархии по времени (функции, вычислимые за большее время, не вычислимы за меньшее) и объясните, почему из неё не следует $P \neq \mathrm{NP}$.
3. Приведите пример «самореференции», дающей нижнюю оценку: опишите машину, которая на входе с кодом M моделирует M с ограничением по времени и отвечает наоборот. Укажите, какое требование ломается, если попытаться отделить этой конструкцией NP от P.